

PokeAPI como grafo: capa de analisis competitivo

Extension del reporte principal hacia el Pokemon competitivo (Smogon singles, gen9 OU). Acá hay consultas de teambuilding sobre el cuadro de tipos y los stats, una capa de meta real de Smogon (usage stats) montada sobre los nodos que ya existen, y tres modelos predictivos evaluados con control de fuga.

Conviene aclarar algo de entrada: el grafo base mide potencial por tipos y stats base, no el meta real (no hay efectos de habilidades, EVs, naturalezas ni items). La capa de Smogon agrega el dato de uso real y cierra parte de esa brecha.

```
In [1]: import math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from neo4j import GraphDatabase
%matplotlib inline
driver = GraphDatabase.driver("bolt://localhost:7687", auth=None)
def q(query):
    with driver.session() as s:
        return pd.DataFrame([r.data() for r in s.run(query)])
print("conectado")
```

conectado

1. Consultas de teambuilding (sin datos externos)

Explotan el cuadro de tipos (EFFECTIVENESS), los stats base y los movepools. Ninguna de las 9 consultas estructurales del reporte principal toca la mecanica de combate.

Mejores tipados defensivos (mas resistencias e inmunidades)

El multiplicador frente a cada tipo atacante es el producto de los factores sobre los 1-2 tipos.

```
In [2]: q("""
MATCH (p:Pokemon)-[:HAS_TYPE]->(def:Type) WHERE p.is_default
WITH p, collect(def) AS defs
MATCH (atk:Type)
OPTIONAL MATCH (atk)-[e:EFFECTIVENESS]->(d) WHERE d IN defs
WITH p, atk, reduce(f=1.0, x IN collect(e.factor/100.0) | f*x) AS mult
RETURN p.identifiaer AS pokemon,
       count(CASE WHEN mult=0.0 THEN 1 END) AS inmunidades,
```

```
count(CASE WHEN mult>0 AND mult<1 THEN 1 END) AS resistencias,
count(CASE WHEN mult>1 THEN 1 END) AS debilidades
ORDER BY (inmunidades+resistencias) DESC, debilidades ASC LIMIT 12
""")
```

Out[2]:

	pokemon	inmunidades	resistencias	debilidades
0	magnemite	1	11	3
1	togedemaru	1	11	3
2	magnezone	1	11	3
3	magneton	1	11	3
4	gholdengo	3	9	4
5	doublade	3	9	4
6	honedge	3	9	4
7	aegislash-shield	3	9	4
8	magearna	2	9	2
9	tinkaton	2	9	2
10	klefki	2	9	2
11	mawile	2	9	2

¿Que amenazas son mas dificiles de murar por tipos?

Un counter resiste todos los STAB del threat. Menos counters = mas dificil de contrarrestar. (Dato verificado en el camino: la intuicion de que Garchomp no tiene counters es falsa; los duales como Skarmory steel/flying resisten ground y dragon a la vez.)

In [3]:

```
q("""
UNWIND ['dragapult','kingambit','garchomp','dragonite','tyranitar','great-tu
MATCH (threat:Pokemon {identifier:name})-[:HAS_TYPE]->(tt:Type)
WITH name, collect(DISTINCT tt) AS T
MATCH (c:Pokemon)-[:HAS_TYPE]->(ct:Type) WHERE c.is_default
WITH name, T, c, collect(ct) AS cdefs
WHERE ALL(tk IN T WHERE
    reduce(f=1.0, d IN cdefs | f * coalesce([(tk)-[e:EFFECTIVENESS]->(d) |
RETURN name AS amenaza, count(c) AS counters_por_tipos ORDER BY counters_por
""")
```

Out[3]: **amenaza** **counters_por_tipos**

0	dragapult	10
1	garchomp	13
2	kingambit	26
3	dragonite	56
4	tyranitar	68
5	great-tusk	131

Revenge-killers: lentos, fuertes, con prioridad STAB de daño

```
In [4]: q("""
MATCH (p:Pokemon)-[:CAN_LEARN]->(m:Move)-[:MOVE_TYPE]->(mt:Type)
WHERE m.priority >= 1 AND m.power > 0 AND p.is_default
MATCH (p)-[:HAS_TYPE]->(pt:Type) WHERE pt = mt
MATCH (p)-[:HAS_STAT]->(:Stat {identifier:'speed'})
MATCH (p)-[:HAS_STAT]->(:Stat {identifier:'attack'})
WITH p.identifiier AS poke, hs.base_stat AS speed, ha.base_stat AS ataque, co
WHERE speed <= 60 AND ataque >= 110
RETURN poke, speed, ataque, prio_stab ORDER BY ataque DESC LIMIT 10
""")
```

Out[4]: **poke** **speed** **ataque** **prio_stab**

0	conkeldurr	45	140	[upper-hand, mach-punch]
1	crabominable	43	132	[upper-hand]
2	machamp	55	130	[vacuum-wave, mach-punch]
3	brute-bonnet	55	127	[sucker-punch]
4	golisopod	40	125	[first-impression]
5	crawdaunt	55	120	[aqua-jet]
6	hariyama	50	120	[vacuum-wave, upper-hand]
7	grimmsnarl	60	120	[sucker-punch]
8	cacturne	55	115	[sucker-punch]
9	doublade	35	110	[shadow-sneak]

Calculadora de daño en Cypher puro

La formula completa de daño vive en una consulta. Charizard Flamethrower vs Venusaur da 208-246, y con el redondeo por etapas (floor tras el roll, pokeRound en el STAB, floor en la efectividad) calza con el damage calc oficial de Pokemon Showdown.

```
In [5]: q("""
MATCH (atk:Pokemon {identifier:'charizard'})-[ha:HAS_STAT]->(:Stat {identifi
MATCH (def:Pokemon {identifier:'venusaur'})-[hd:HAS_STAT]->(:Stat {identifie
MATCH (m:Move {identifier:'flamethrower'})-[:MOVE_TYPE]->(mt:Type)
WITH atk, m, mt, ha, hd, CASE WHEN exists((atk)-[:HAS_TYPE]->(mt)) THEN 1.5
MATCH (def:Pokemon {identifier:'venusaur'})-[:HAS_TYPE]->(dt:Type)
OPTIONAL MATCH (mt)-[e:EFFECTIVENESS]->(dt)
WITH m, ha, hd, stab, reduce(eff=1.0, f IN collect(coalesce(e.factor,100)/10
WITH m, stab, typeMult,
    toInteger(floor((2.0*ha.base_stat + 31 + floor(252/4.0)))) + 5 AS A,
    toInteger(floor((2.0*hd.base_stat + 31 + floor(252/4.0)))) + 5 AS D
WITH m.power AS power, stab, typeMult,
    toInteger(floor(floor(floor(42.0*m.power*A/toFloat(D)) / 50))) + 2 AS b
RETURN power, stab, typeMult,
    toInteger(floor(floor(toFloat(floor(baseDmg*0.85))*stab + 0.5)*typeMu
    toInteger(floor(floor(toFloat(baseDmg)*stab + 0.5)*typeMult)) AS dmg_
""")
```

```
Out[5]:
```

	power	stab	typeMult	dmg_min	dmg_max
0	90	1.5	2.0	208	246

2. Capa de meta real (Smogon usage stats)

Cargamos las usage stats mensuales de Smogon (gen9 OU, chaos.json) como subgrafo sobre los nodos que ya existen: `(Pokemon)-[:USED_IN]->(:Format)`, `RUNS_MOVE`, `HOLDS_ITEM`, `USES_ABILITY`, `TEAMMATE_OF`, `CHECKED_BY`. Se carga con `python pipeline/06_smogon.py`.

Kit legal (aprendible) vs kit realmente usado en el meta

```
In [6]: q("""
MATCH (p:Pokemon)-[u:USED_IN]->(:Format {tier:'gen9ou'})
OPTIONAL MATCH (p)-[:CAN_LEARN]->(legal:Move)
WITH p, u.usage AS uso_pct, count(DISTINCT legal) AS aprendibles
OPTIONAL MATCH (p)-[:RUNS_MOVE]->(usado:Move)
RETURN p.identifier AS pokemon, uso_pct, aprendibles, count(DISTINCT usado)
ORDER BY uso_pct DESC LIMIT 12
""")
```

Out[6]:

	pokemon	uso_pct	aprendibles	usados_en_meta
0	great-tusk	33.727	58	13
1	zamazenta	28.680	68	12
2	gholdengo	27.633	45	11
3	dragonite	25.346	138	12
4	kingambit	25.270	58	5
5	gliscor	19.386	110	12
6	ting-lu	17.423	46	11
7	kyurem	17.035	86	13
8	ogerpon-wellspring-mask	16.466	62	14
9	dragapult	14.587	69	16
10	samurott-hisui	14.394	66	12
11	raging-bolt	14.264	55	11

Que tipos dominan el meta de OU

In [7]:

```
q("""  
MATCH (p:Pokemon)-[u:USED_IN]->(:Format {tier:'gen9ou'})  
MATCH (p)-[:HAS_TYPE]->(t:Type)  
RETURN t.identifrier AS tipo, count(DISTINCT p) AS mons_en_OU, round(avg(u.uso_pct), 2) AS uso_promedio  
ORDER BY mons_en_OU DESC LIMIT 10  
""")
```

Out[7]:

	tipo	mons_en_OU	uso_promedio
0	water	39	2.13
1	psychic	34	1.28
2	fighting	27	3.28
3	dark	27	3.10
4	grass	26	1.47
5	flying	26	3.71
6	steel	26	3.52
7	fire	23	2.31
8	dragon	21	4.41
9	poison	21	3.00

3. Modelos predictivos (evaluacion adversarial)

Tres modelos, todos con control de fuga: el **label shuffleado** debe dar AUC ~0.5 (si diera alto, habria fuga). Las metricas se miden **out-of-fold** (cross-validation), nunca sobre el train.

3.1 ¿Se puede predecir si un Pokemon se usa en OU?

Ablacion: baseline (solo BST + legendario) vs + features de grafo. Y negativos **difíciles** (solo fully-evolved no-legendarios) para no medir lo trivial (legendario vs pre-evo).

```
In [8]: from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_predict, StratifiedKFold
from sklearn.metrics import roc_auc_score, roc_curve, precision_recall_curve

stats = q("MATCH (p:Pokemon {is_default:true})-[r:HAS_STAT]->(s:Stat) RETURN
stats[\"bst\"] = stats.sum(axis=1)
meta = q("""MATCH (p:Pokemon {is_default:true})-[:IS_SPECIES]->(sp:Species)
OPTIONAL MATCH (p)-[:CAN_LEARN]->(mv:Move) WITH p,sp,count(DISTINCT mv) AS m
OPTIONAL MATCH (p)-[:HAS_ABILITY]->(ab:Ability) WITH p,sp,movepool,count(DIS
OPTIONAL MATCH (p)-[:CAN_LEARN]->(pm:Move) WHERE pm.priority>0 AND pm.power>
RETURN p.id AS id, movepool, n_abil, count(DISTINCT pm) AS prio_moves,
CASE WHEN sp.is_legendary THEN 1 ELSE 0 END AS legendary,
CASE WHEN sp.is_mythical THEN 1 ELSE 0 END AS mythical,
CASE WHEN (sp)-[:EVOLVES_TO]->(:Species) THEN 0 ELSE 1 END AS fully_evolve
mt = q("""MATCH (p:Pokemon {is_default:true})-[:HAS_TYPE]->(def:Type) WITH p
MATCH (atk:Type) OPTIONAL MATCH (atk)-[e:EFFECTIVENESS]->(d) WHERE d IN defs
WITH p, atk, reduce(f=1.0,x IN collect(e.factor/100.0)|f*x) AS m
RETURN p.id AS id, sum(CASE WHEN m>1 THEN 1 ELSE 0 END) AS weak,
sum(CASE WHEN m<1 AND m>0 THEN 1 ELSE 0 END) AS resist, sum(CASE WHEN m=0
# label a nivel de ESPECIE: viable si CUALQUIER forma se usa (landorus-theri
lab = q("""MATCH (p:Pokemon {is_default:true})-[:IS_SPECIES]->(sp:Species)
RETURN p.id AS id, CASE WHEN EXISTS { (sp)-[:IS_SPECIES]->(:Pokemon)-[:USED_
D = stats.join([meta, mt, lab], how="inner").fillna(0)
base = ["bst","legendary"]
full = ["bst","hp","attack","defense","special-attack","special-defense","sp
def oof(X,y,seed=42):
    return cross_val_predict(RandomForestClassifier(400,class_weight="balanc
hard = D[(D.in_ou==1)|((D.fully_evolved==1)&(D.legendary==0)&(D.mythical==0)
yh = hard["in_ou"].astype(int)
# AUC media +/- desviacion sobre 10 semillas: el intervalo dice si el salto
def ci(cols): a=[roc_auc_score(yh,oof(hard[cols],yh,seed=k)) for k in range(
mb,sb=ci(base); mf,sf=ci(full)
print(f"{len(hard)} pokemon comparables, {int(yh.sum())} en OU")
print(f"baseline (BST+legendario): AUC={mb:.3f}+/-{sb:.3f}")
print(f"+ features de grafo: AUC={mf:.3f}+/-{sf:.3f} (delta={mf-mb:+.
nulo=np.mean([roc_auc_score(p, oof(hard[full],p,seed=k)) for k in range(5) f
print(f"control de fuga (label shuffleado): AUC={nulo:.3f} (debe ser ~0.5)")
fig,ax=plt.subplots(1,2,figsize=(12,5))
for cols,labn,c in [(base,"baseline (BST+leg.)","#8172b3"),(full,"+ grafo",
pr=oof(hard[cols],yh); fpr,tpr,_=roc_curve(yh,pr); prec,rec,_=precision_
ax[0].plot(fpr,tpr,color=c,label=f"{labn}: AUC={roc_auc_score(yh,pr):.3f
```

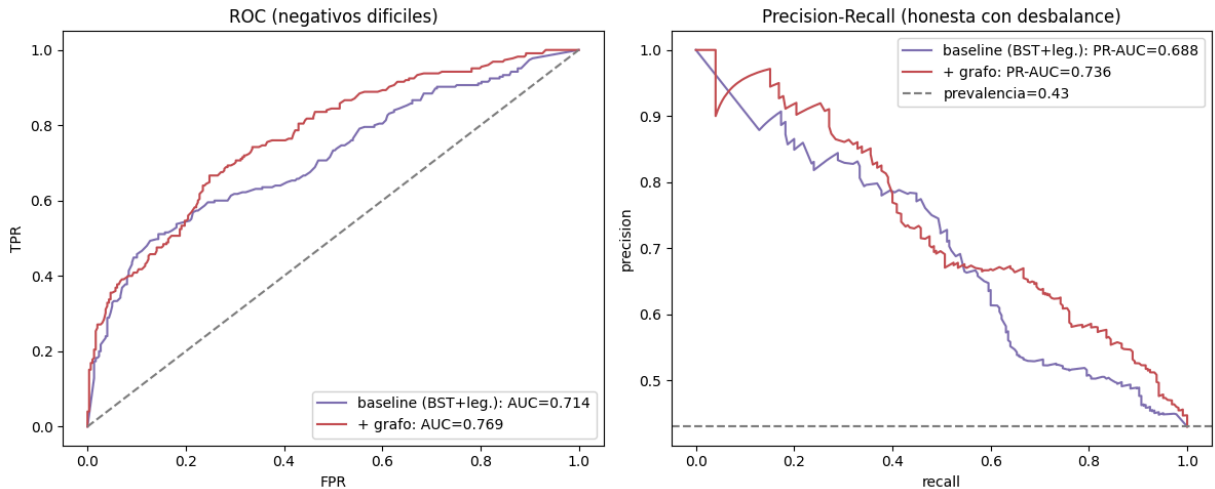
```
ax[1].plot(rec,prec,color=c,label=f"{labn}: PR-AUC={average_precision_sc
ax[0].plot([0,1],[0,1],"--",color="gray"); ax[0].legend(loc="lower right");
ax[1].axhline(yh.mean(),ls="--",color="gray",label=f"prevalencia={yh.mean()}:
plt.tight_layout(); plt.show()
```

523 pokemon comparables, 225 en OU

baseline (BST+legendario): AUC=0.707+/-0.013

+ features de grafo: AUC=0.777+/-0.008 (delta=+0.070)

control de fuga (label shuffleado): AUC=0.512 (debe ser ~0.5)



El baseline BST+legendario da AUC ~0.71; sumar features de grafo (movepool, distribucion de stats, matchups) sube a ~0.77. El delta (~+0.07) queda fuera del intervalo de las semillas (~0.01), así que es real, y el shuffle (~0.5) confirma que no es fuga. La PR-AUC, más honesta con el desbalance, acompaña: sube de ~0.70 a ~0.74.

3.2 Recomendacion de teammates: ¿que predice el co-uso real?

Link prediction sobre `TEAMMATE_OF`. Contrastamos popularidad (ambos se usan mucho) vs la complementariedad defensiva calculada en nuestro grafo de tipos vs la topología de co-ocurrencia.

```
In [9]: from sklearn.model_selection import train_test_split
tm = q("MATCH (a:Pokemon)-[t:TEAMMATE_OF]->(b:Pokemon) RETURN a.id AS a, b.id AS b")
usage = q("MATCH (p:Pokemon)-[u:USED_IN]->(:Format {tier:'gen9ou'}) RETURN p.id AS p, u AS u")
nodeset = set(usage); nodes = sorted(nodeset); nl = np.array(nodes)
prof = q("MATCH (p:Pokemon)-[:USED_IN]->(:Format {tier:'gen9ou'}) MATCH (p WITH p, collect(def) AS defs MATCH (atk:Type) OPTIONAL MATCH (atk)-[e:EFFECT]->(def) WITH p, atk, reduce(f=1.0,x IN collect(e.factor/100.0)|f*x) AS m RETURN p.id AS id, [a IN collect(CASE WHEN m>=2 THEN atk.identifier END) WHERE a IN collect(CASE WHEN m<1 THEN atk.identifier END) WHERE a IS NOT NULL] AS weak")
weak={r.id:set(r.weak) for r in prof.itertuples()}; resist={r.id:set(r.resist) for r in prof.itertuples()}
edge_set={(min(a,b),max(a,b)) for a,b in zip(tm.a,tm.b) if a in nodeset and b in nodeset}
pos=np.array(list(edge_set))
def comp(u,v):
    wu,wv,ru,rv=weak.get(u,set()),weak.get(v,set()),resist.get(u,set()),resist.get(v,set())
    return ((len(wu&rv)/len(wu) if wu else 1)+(len(wv&ru)/len(wv) if wv else 1))/2
def negs(k,rng):
```

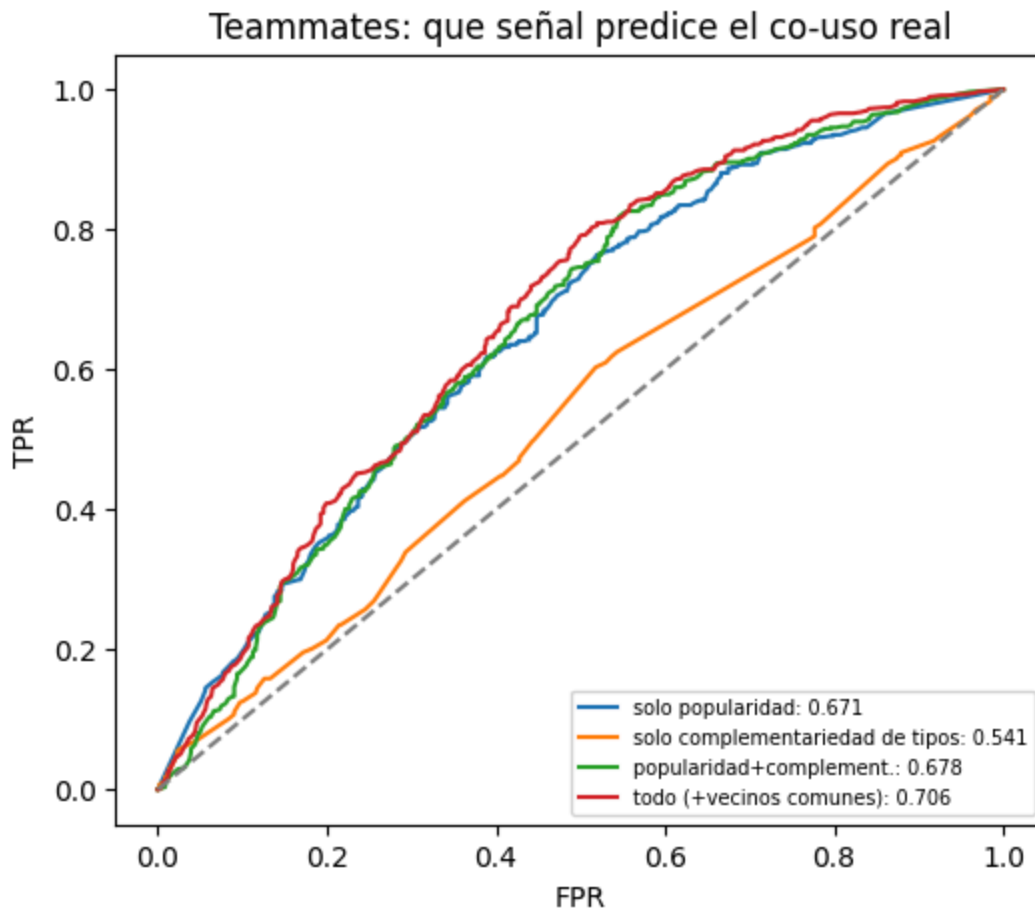
```

o=set()
while len(o)<k:
    u,v=rng.choice(nl,2,replace=False); e=(int(min(u,v)),int(max(u,v)))
    if e not in edge_set: o.add(e)
return np.array(list(o))
def feats(P,adj): return np.array([[usage.get(u,0)*usage.get(v,0), comp(u,v)
COLS={"solo popularidad":[0],"solo complementariedad de tipos":[1],"populari
def run(seed):
    rng=np.random.default_rng(seed); neg=negs(len(pos),rng)
    ptr,pte=train_test_split(pos,test_size=0.25,random_state=seed); ntr,nte=
    adj={n:set() for n in nodes} # adyacencia SÓLO con train: si entran ar
    for a,b in ptr: a,b=int(a),int(b); adj[a].add(b); adj[b].add(a)
    ytr=np.r_[np.ones(len(ptr)),np.zeros(len(ntr))]; yte=np.r_[np.ones(len(p
    Xtr,Xte=feats(np.vstack([ptr,ntr]),adj),feats(np.vstack([pte,nte]),adj)
    o={}
    for nombre,idx in COLS.items():
        m=RandomForestClassifier(300,random_state=seed,n_jobs=-1).fit(Xtr[:,
        o[nombre]=(roc_auc_score(yte,pr), roc_curve(yte,pr))
        msh=RandomForestClassifier(200,random_state=seed,n_jobs=-1).fit(Xtr,rng.
    return o
res=[run(k) for k in range(10)]
print(f"{len(nodes)} mons OU, {len(pos)} aristas teammate. AUC holdout 75/25
plt.figure(figsize=(6,5))
for nombre in COLS:
    aucs=[r[nombre][0] for r in res]; print(f" {nombre:32s} AUC={np.mean(au
    fpr, tpr, _=res[0][nombre][1]; plt.plot(fpr,tpr,label=f"{nombre}: {np.mear
    print(f" control de fuga (label shuffleado): AUC={np.mean([r['_nulo'] for
    plt.plot([0,1],[0,1],"--",color="gray"); plt.legend(fontsize=7,loc="lower ri
    plt.title("Teammates: que señal predice el co-uso real"); plt.show()

```

238 mons OU, 2348 aristas teammate. AUC holdout 75/25, media+/-std de 10 splits:

solo popularidad	AUC=0.671+/-0.014
solo complementariedad de tipos	AUC=0.541+/-0.010
popularidad+complement.	AUC=0.678+/-0.016
todo (+vecinos comunes)	AUC=0.706+/-0.015
control de fuga (label shuffleado):	AUC=0.500 (debe ser ~0.5)



Hallazgo adversarial: la complementariedad defensiva de tipos **sola** apenas supera el azar (~ 0.54) y no mejora sobre popularidad. La idea intuitiva de que los buenos compañeros se cubren las debilidades de tipo **no** explica el co-uso real; lo que lo predice es la co-ocurrencia (vecinos comunes, ~ 0.71), y se mantiene despues de cerrar la fuga topologica (la adyacencia se arma solo con train). El grafo de tipos describe el juego, no el meta.

Red de teammates como grafo

Los mismos datos de co-uso vistos como red. Cada arista es una dupla que aparece junta seguido; las comunidades por modularidad son los cores de equipo que el meta arma solo (uno defensivo de gliscor/corviknight/zamazenta, otro ofensivo de kingambit/great-tusk, etc).

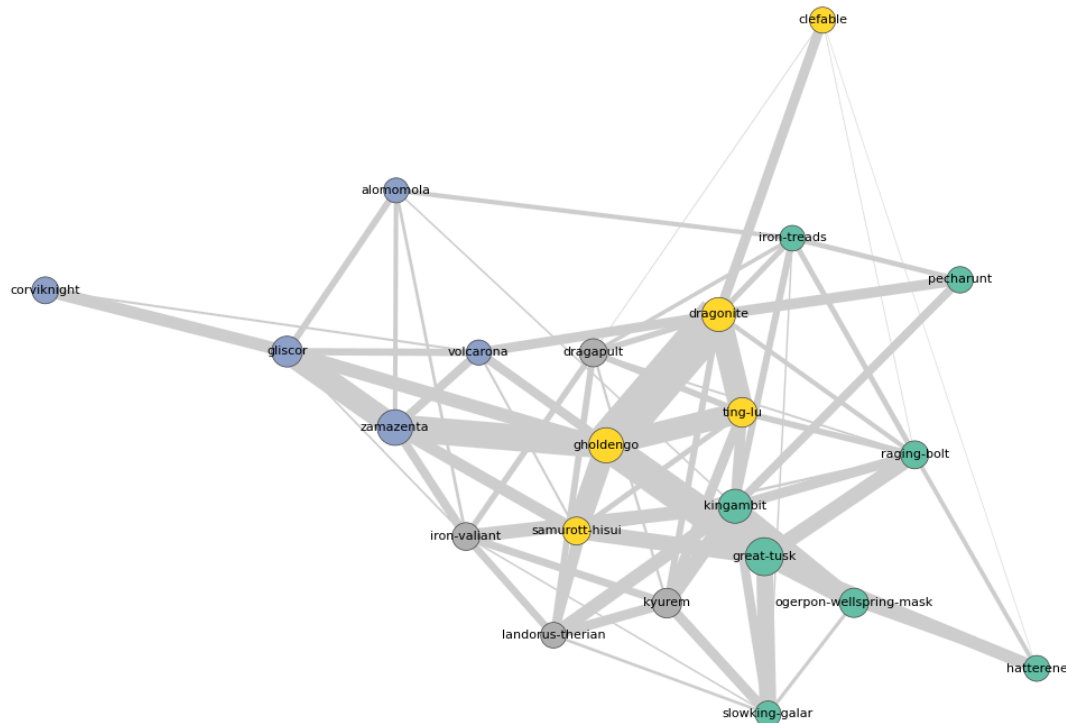
```
In [10]: import networkx as nx
from networkx.algorithms.community import greedy_modularity_communities
top = q("MATCH (p:Pokemon)-[u:USED_IN]->(t:Pokemon) RETURN p.id, t.id, u")
S=set(top["id"]); um=dict(zip(top["id"],top["use"]))
te=q("MATCH (a:Pokemon)-[t:TEAMMATE_OF]->(b:Pokemon) RETURN a.id, b.id, t")
te=te[te["s"].isin(S)&te["t"].isin(S)]
H=nx.Graph(); H.add_nodes_from(S)
for _,r in te.iterrows():
    if H.has_edge(r["s"],r["t"]): H[r["s"]][r["t"]]["w"]=max(H[r["s"]][r["t"]].get("w",0),um[r["s"]][r["t"]])
```

```

else: H.add_edge(r["s"],r["t"],w=r["w"])
comms=list(greedy_modularity_communities(H,weight="w")); cmap={n:i for i,c in enumerate(comms)}
posg=nx.spring_layout(H,k=0.9,seed=7,weight="w",iterations=200)
plt.figure(figsize=(11,8))
nx.draw_networkx_edges(H,posg,width=[0.3+0.012*H[u][v]["w"] for u,v in H.edges])
nx.draw_networkx_nodes(H,posg,node_size=[120+14*um.get(n,0) for n in H.nodes])
nx.draw_networkx_labels(H,posg,font_size=8)
plt.title(f"Red de teammates en gen9 OU ({len(comms)} comunidades por modularidad)")

```

Red de teammates en gen9 OU (4 comunidades por modularidad, tamaño=uso)



3.3 Clustering de roles por stats base

No supervisado sobre las proporciones de los 6 stats (forma del spread, no poder bruto), para que un frail-veloz y un muro-lento caigan en clusters distintos sin importar su BST.

```

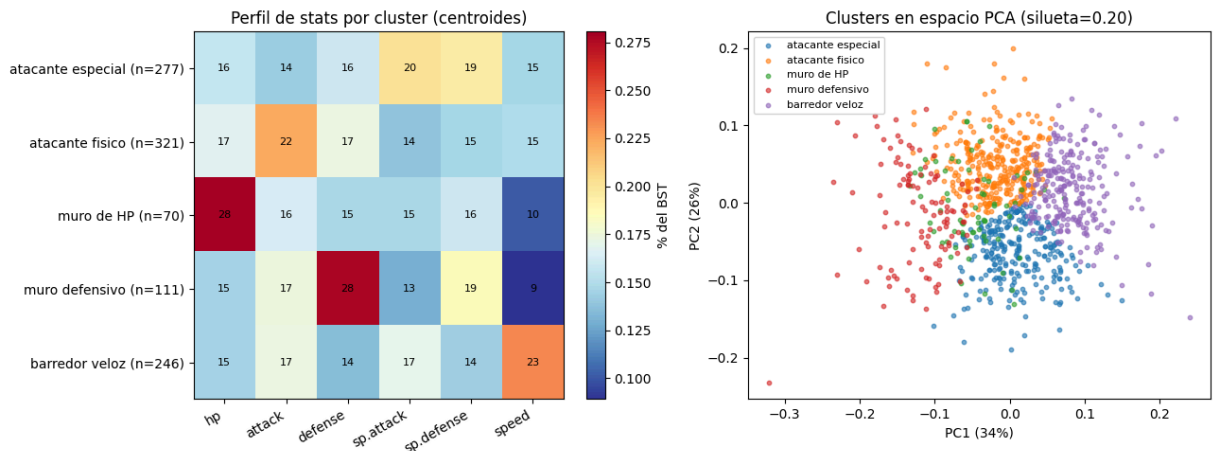
In [11]: from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA
so=["hp","attack","defense","special-attack","special-defense","speed"]
R=q("MATCH (p:Pokemon {is_default:true})-[r:HAS_STAT]->(s:Stat) RETURN p.id
prop=R.div(R.sum(axis=1),axis=0); Xc=prop.values
km=KMeans(5,random_state=42,n_init=10).fit(Xc); R["cluster"]=km.labels_
sil=silhouette_score(Xc,km.labels_)
cent=prop.groupby(km.labels_).mean()
ejes=pd.DataFrame({"speed":cent["speed"],"ofensa":cent["attack"]+cent["speci
z=(ejes-ejes.mean())/ejes.std()
def lab(c):

```

```

dom=z.loc[c].idxmax(); fis=cent.loc[c,"attack"]>=cent.loc[c,"special-att
if dom=="speed": return "barredor veloz" if ejes.loc[c,"ofensa"]>=ejes.m
if dom=="bulk":
    if z.loc[c,"speed"]>=0: return "tanque"
    return "muro de HP" if cent.loc[c,"hp"]>=cent.loc[c,"defense"] else
return "atacante fisico" if fis else "atacante especial"
rol={c:lab(c) for c in cent.index}; R["rol"]=R["cluster"].map(rol)
fig,ax=plt.subplots(1,2,figsize=(13,5))
im=ax[0].imshow(cent.values,cmap="RdYlBu_r",aspect="auto")
ax[0].set_xticks(range(6)); ax[0].set_xticklabels([s.replace("special-", "sp.
ax[0].set_yticks(range(5)); ax[0].set_yticklabels([f"{rol[c]} (n={km.labels
for i,c in enumerate(cent.index):
    for j in range(6): ax[0].text(j,i,f"{cent.values[i,j]*100:.0f}",ha="cent
fig.colorbar(im,ax=ax[0],label="% del BST"); ax[0].set_title("Perfil de stat
P=PCA(2).fit(Xc); Y=P.transform(Xc)
for c in cent.index:
    m=km.labels_==c; ax[1].scatter(Y[m,0],Y[m,1],s=10,alpha=0.6,label=rol[c]
ax[1].set_xlabel(f"PC1 ( {P.explained_variance_ratio_[0]:.0%})"); ax[1].set_y
ax[1].legend(fontsize=8); ax[1].set_title(f"Clusters en espacio PCA (silueta
plt.tight_layout(); plt.show()
print(R["rol"].value_counts().to_string())

```



```

rol
atacante fisico      321
atacante especial    277
barredor veloz       246
muro defensivo       111
muro de HP           70

```

El heatmap de centroides hace legible cada rol: un cluster con ~28% del BST en HP y ~10% en velocidad es un muro, otro con ~23% en velocidad es un barredor, etc. El PCA muestra lo honesto del asunto: los arquetipos son un continuo, no islas separadas (la silueta ronda 0.2). Sirven como esqueleto descriptivo, no como tier de Smogon: el rol meta real depende del typing, los items y los EVs que el grafo base no tiene.

4. Como se verifica que las metricas son correctas

- **Out-of-fold:** en viabilidad cada Pokemon lo predice un modelo de un fold que NO lo vio (`cross_val_predict`), nunca se mide sobre el train. Teammates usa un

holdout 75/25 repetido.

- **Intervalos:** cada AUC se promedia sobre varias particiones (media +/- desviacion), para no leer un numero de una sola semilla. Un salto que cae fuera del intervalo es real, no ruido.
- **Control de fuga del label:** con el label permutado al azar el AUC cae a ~0.5 (viabilidad ~0.51, teammates ~0.50). Si hubiera fuga del label en alguna feature, seguiria alto.
- **Sin fuga topologica:** en teammates la feature de vecinos comunes se calcula con la adyacencia de ENTRENAMIENTO, no la del grafo completo; de lo contrario el modelo veria las aristas que debe predecir.
- **Baselines y ablacion:** cada modelo se compara contra un baseline trivial (BST, popularidad) para aislar cuanto aporta el grafo, y se reporta PR-AUC ademas de ROC en las tareas desbalanceadas.
- **Negativos dificiles:** en viabilidad los negativos son Pokemon comparables (fully-evolved no legendarios), no pre-evos triviales, para que el AUC no infle midiendo lo obvio.

```
In [12]: driver.close()  
print("fin del reporte competitivo")
```

fin del reporte competitivo